SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: _____

Assessment Tasks
Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.
   Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

SET 2:

1.A retail company collects terabytes of customer transaction data every day from online and offline stores. The management wants to analyze purchasing patterns in near real time to improve product recommendations and inventory management.

- Design a cloud-based big data architecture using distributed storage and processing frameworks.
- Explain how batch processing and stream processing can be integrated.
- Discuss how data lakes and data warehouses can be used together.
- Recommend techniques to ensure scalability and cost efficiency.

2.A healthcare organization is migrating patient records and medical imaging data to the cloud. The solution must comply with strict privacy regulations while allowing authorized doctors to access records securely from multiple locations.

- Design a secure cloud architecture that protects sensitive healthcare data.
- Explain how encryption, identity and access management (IAM), and key management should be implemented.
- Discuss auditing, logging, and compliance monitoring strategies.
- Recommend methods to prevent unauthorized access and data breaches.

3. An online ticket booking platform experiences sudden spikes in traffic whenever popular events are announced. The company wants to reduce infrastructure management while ensuring fast response times.

- Design a serverless cloud architecture for the application.
- Explain how event-driven computing can improve scalability.
- Discuss the role of managed databases, API gateways, and message queues.
- Evaluate the advantages and limitations of serverless computing for this workload.

4. A multinational company gathers data from sales, finance, manufacturing, and customer support systems. The organization requires a centralized analytics platform with high-quality and consistent data.

- Design a cloud-based ETL/ELT pipeline for integrating data from multiple sources.
- Explain how data quality, metadata management, and governance should be maintained.
- Discuss the role of workflow orchestration and scheduling tools.
- Recommend strategies for optimizing query performance and storage costs.

5. A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. The analytics platform must support continuous model training and real-time predictions.

- Design a cloud-based machine learning and analytics architecture.
- Explain how big data technologies can support model training and inference.
- Discuss the use of cloud-native AI/ML services for model deployment and monitoring.
- Recommend strategies for handling large-scale data ingestion, feature engineering, and model updates.

SET 3 :

1. A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

- Design a cloud migration strategy suitable for the organization.
- Explain how different migration approaches (Rehost, Refactor, Replatform, Repurchase, Retire, Retain) can be applied.
- Discuss methods to validate application performance after migration.
- Recommend strategies to reduce migration risks and downtime.

2. A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

- Design a cloud-based architecture for IoT data collection and analytics.
- Explain how message brokers, stream processing, and cloud storage are integrated.
- Discuss methods for handling high-volume, high-velocity sensor data.
- Recommend techniques to ensure scalability, reliability, and security.

3. A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

- Design a cloud-based business intelligence architecture.
- Explain how data visualization tools integrate with cloud data warehouses.
- Discuss techniques for optimizing dashboard performance with large datasets.
- Recommend security measures to protect business reports and sensitive data.

4. An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

- Design a cloud monitoring and performance optimization solution.

- Explain how cloud monitoring tools, metrics, logs, and tracing help identify performance bottlenecks.
- Discuss strategies for resource optimization and capacity planning.
- Recommend methods to improve application availability while controlling operational costs.

5. A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

- Design a cloud-based data lifecycle management strategy.
- Explain how data classification, retention policies, and archival storage should be implemented.
- Discuss the importance of governance, auditing, and regulatory compliance in big data environments.
- Recommend techniques for balancing data accessibility, security, and cost optimization.

Question 1: Video Streaming Platform for Millions of Concurrent Users

1.1 Understanding the Scenario and Key Issues

A media company is moving its streaming service to the cloud. The platform must serve millions of simultaneous viewers with minimal buffering, particularly during live events and new-release launches when demand rises sharply within minutes. The principal risks are: (i) origin servers overwhelmed by simultaneous requests, (ii) high latency for viewers far from the data centre, (iii) bandwidth cost, because video is the largest egress workload on the internet, (iv) uneven demand between peak evenings and quiet daytime hours, and (v) content piracy and unauthorised access. The architecture must therefore be elastic, heavily cached close to users, and cost-aware.

1.2 Proposed Cloud Architecture

The design is organised in layers, each using scalable cloud services:

- **Ingest and transcoding layer:** uploaded or live source video lands in object storage (for example Amazon S3). A pool of elastic workers, or a managed media service such as AWS Elemental MediaConvert, transcodes each title into multiple resolutions and bitrates and packages it as HLS or MPEG-DASH segments of two to six seconds. Live streams use a similar pipeline with low-latency packaging.
- **Origin storage layer:** the packaged segments and manifests are stored in durable, replicated object storage. Origin servers sit behind a regional load balancer and are shielded by the CDN.
- **Content delivery layer:** a multi-tier CDN (edge locations, regional edge caches and an origin shield) delivers segments to users. Popular titles are pre-positioned (pre-warmed) in edge caches before a release.
- **Application layer:** stateless microservices (user profiles, catalogue, search, recommendations, playback authorisation) run in containers or auto-scaling groups across several Availability Zones behind an application load balancer and API gateway.
- **Data layer:** a managed NoSQL database (such as DynamoDB or Cassandra) for profiles and watch history, a relational database with read replicas for billing, and an in-memory cache (Redis or Memcached) for sessions, catalogue metadata and frequently read data.
- **Security and observability layer:** digital rights management (DRM) and signed URLs or tokens for content protection, a web application firewall, and a monitoring stack for metrics, logs and alerts.

1.3 Scalability and Caching Techniques

Caching is the most important technique for streaming, because the same video segments are requested by thousands of viewers. The first request for a segment travels to the origin, but the CDN stores it and serves all later requests from the edge, so more than 90 to 95 percent of traffic never reaches the origin. Long time-to-live values are used for immutable video segments, while short values are used for live manifests that change every few seconds. Request collapsing at the origin shield ensures that when thousands of viewers ask for a new live segment at once, only one request reaches the origin. At the application tier, Redis caches metadata and personalised recommendations, which protects databases from repeated reads. Browsers and players also buffer ahead, and adaptive bitrate (ABR) streaming switches between quality levels according to measured bandwidth, so that the stream continues smoothly on slower connections.

Scalability is achieved by keeping every tier stateless and horizontally scalable: user sessions are held in the cache or in signed tokens rather than on individual servers, so any instance can handle any request, and the databases are partitioned (sharded) by user or title identifier.

1.4 Load Balancing and Auto-Scaling for Smooth Streaming

Global traffic management (for example latency-based DNS routing) sends each viewer to the nearest healthy region and CDN edge. Within a region, application load balancers distribute API and playback-authorisation requests across healthy instances using health checks, so a failed server is removed from rotation within seconds. Auto-scaling adds capacity before the user experience degrades:

- **Scheduled scaling** raises capacity before announced premieres or sports events.
- **Target-tracking scaling** keeps CPU, requests per second or concurrent sessions per instance near a target (for example 60 percent CPU) and adds instances when the metric rises.
- **Queue-based scaling** is used for the transcoding workers, which scale according to the number of jobs waiting in the queue.
- **Pod and node autoscaling** in Kubernetes (Horizontal Pod Autoscaler and cluster autoscaler) handles containerised microservices.

Together these mechanisms mean viewers experience fast start-up and few rebuffering events: the CDN removes the bandwidth bottleneck, and auto-scaling and load balancing remove the compute bottleneck in the control path.

1.5 Cost Optimisation Strategies

Video delivery costs are dominated by data transfer, storage and compute, so each is addressed separately:

Cost area	Optimisation strategy
Data transfer	Maximise CDN cache hit ratio; negotiate committed-use CDN pricing or use multiple CDNs and route traffic to the cheapest one that meets quality targets; use efficient codecs (H.265/HEVC, AV1) to cut bitrate by 30 to 50 percent at the same quality.
Compute	Use reserved instances or savings plans for the predictable baseline, on-demand for normal peaks, and spot instances for fault-tolerant jobs such as transcoding; right-size instances with monitoring data.
Storage	Lifecycle policies move rarely watched titles to infrequent-access or archive tiers; keep only the bitrate ladders that are actually used; delete temporary files automatically.
Scaling	Scale in aggressively after peaks so that no idle capacity is paid for; use scheduled scaling to avoid over-provisioning.
Governance	Tag resources by service and team, set budgets and anomaly alerts, and review cost dashboards regularly (FinOps practice).

1.6 Justification of Decisions

A CDN-centred architecture is chosen because it directly addresses the dominant bottleneck, which is distance and bandwidth, and it lowers origin cost. Stateless microservices and managed databases

are chosen because they scale horizontally without manual intervention. Using spot and reserved capacity together balances cost with reliability. The principal remaining risks, such as a CDN outage, are mitigated by a multi-CDN strategy, and cost overruns by budgets and alerts.

Question 2: Containers Across Multiple Cloud Providers for a Fintech Startup

2.1 Understanding the Scenario and Key Issues

A fintech startup runs containerised applications on more than one cloud provider, perhaps to avoid vendor lock-in, meet regional regulations, or improve resilience. The challenges it faces are typical of multi-cloud: environments drift apart because each provider has different networking, identity and storage services; deployments are done manually or by different scripts on each cloud; releases are inconsistent; and it is hard to see and control what is running where. In a financial setting there is the additional requirement for security, auditability and high availability.

2.2 Proposed Solution: Orchestration and Multi-Cloud Strategy

- **Standardise on containers and Kubernetes.** Every service is packaged as an OCI-compliant container image and deployed to Kubernetes clusters, using managed services (Amazon EKS, Azure AKS, Google GKE) on each provider. Kubernetes provides the same API everywhere, which removes most differences between clouds.
- **Single image registry and supply-chain controls.** Images are built once in a CI pipeline, scanned for vulnerabilities, signed and promoted through environments, so the identical artefact is deployed to each cloud.
- **Infrastructure as Code.** Terraform or Pulumi defines clusters, networks and IAM roles in version-controlled code, so every environment is reproducible and reviewed.
- **GitOps deployment.** Tools such as Argo CD or Flux continuously compare the desired state in a Git repository with the actual cluster state, apply changes automatically and revert manual drift. Rollbacks are simply Git reverts, and every change is auditable.
- **Fleet management and policy.** A multi-cluster management layer (for example Rancher, Anthos or Azure Arc) and policy engines (OPA Gatekeeper, Kyverno) enforce the same security and configuration rules on all clusters.
- **Unified observability and identity.** Prometheus, Grafana and OpenTelemetry collect metrics, logs and traces from all clouds into one view, and a central identity provider with federated single sign-on and short-lived credentials controls access.

2.3 Service Discovery and Scaling

Inside each cluster, Kubernetes Services give every group of Pods a stable virtual IP and DNS name (for example payments.default.svc), and CoreDNS resolves it, so services find each other without hard-coded addresses even when Pods are replaced. Ingress controllers and API gateways expose services externally. Across clusters and clouds, a service mesh such as Istio or Linkerd, or multi-cluster tools such as Cilium Cluster Mesh, provides cross-cluster service discovery, mutual TLS between services, traffic shifting (canary releases) and failover to a healthy cluster. Global DNS or a global load balancer directs external users to the closest or healthiest cluster.

For scaling, the Horizontal Pod Autoscaler increases or decreases Pod replicas using CPU, memory or custom metrics such as transaction rate, the Vertical Pod Autoscaler tunes resource requests, and the cluster autoscaler (or Karpenter) adds or removes nodes on each provider. Event-driven autoscalers such as KEDA can scale workers by queue length. Resource requests and limits prevent one service from starving others, which is important for payment processing.

2.4 Benefits and Risks of Multi-Cloud

Benefits	Risks
Reduced vendor lock-in and stronger bargaining power on price	Higher operational complexity and need for broader skills
Resilience: workloads can fail over if one provider has an outage	Inconsistent security models, IAM and misconfiguration risk across clouds
Compliance flexibility: use the provider with a region or certification required by a regulator	Data transfer (egress) charges and latency between clouds
Choose best-of-breed services and optimise cost by workload	Using provider-specific services reduces portability and defeats the purpose
Geographic reach for global customers	Harder monitoring, troubleshooting and cost control; larger attack surface

For a startup the principal danger is complexity that exceeds the size of the team. The recommended approach is therefore to keep the platform minimal: use a small number of providers, restrict use of proprietary services to well-isolated components, automate everything through IaC and GitOps, and consider an active-passive rather than active-active design unless the business needs the latter.

2.5 Justification

Kubernetes with GitOps is chosen because it gives one deployment model for all clouds, which directly solves the consistency problem. The service mesh and central observability are justified by the security and audit requirements of financial services. Multi-cloud is accepted only where its resilience and compliance benefits outweigh the extra cost and complexity.

Question 3: Backup and Disaster Recovery for an E-Learning Platform

3.1 Understanding the Scenario and Key Issues

An e-learning platform lost data because someone deleted it accidentally. This shows that high availability and replication alone are not enough: replication copies deletions and corruption to every replica instantly. The platform needs point-in-time recovery, protected copies that cannot be removed easily, and a tested plan. Its data includes course content (videos and documents), user accounts, progress and grades in databases, and uploaded assignments. Key risks are human error, software bugs, ransomware, regional outages and misconfiguration.

3.2 Cloud-Native Backup, Replication and Versioning Design

- **Versioning and soft delete:** object storage buckets (S3, Azure Blob, Cloud Storage) have versioning enabled, so overwriting or deleting an object only creates a new version or a delete marker and earlier versions can be restored. Soft delete or recycle-bin features for databases and virtual disks keep removed items for a retention period.
- **Automated backups and snapshots:** managed databases use automated daily snapshots plus transaction-log archiving for point-in-time recovery (for example RDS can restore to any second within the retention window). Block volumes are snapshotted on a schedule using a central service such as AWS Backup or Azure Backup, which applies uniform policies across services.

- **Replication:** data is replicated synchronously across Availability Zones for high availability (multi-AZ databases, zone-redundant storage) and asynchronously to a second region (cross-region replication, read replicas, geo-redundant storage) for disaster recovery.
- **Protection of backups:** backups are placed in a separate account or vault, encrypted with managed keys, protected with immutability (S3 Object Lock, vault lock or WORM policies) and secured with least-privilege IAM and multi-factor delete, so that a compromised or careless administrator cannot erase them.
- **Lifecycle management:** older backups transition to cheaper archive tiers and are deleted when retention requirements expire.
- **Infrastructure as Code and images:** infrastructure definitions and machine or container images are kept in version control so the environment can be rebuilt quickly.

3.3 Achieving RTO and RPO

The recovery time objective (RTO) is the maximum acceptable time to restore service after an incident, and the recovery point objective (RPO) is the maximum acceptable amount of data loss, measured in time. They determine the cost and design of the strategy.

Strategy	Typical RTO / RPO	How it is achieved
Backup and restore	RTO hours; RPO hours to a day	Scheduled backups restored into new infrastructure; lowest cost
Pilot light	RTO tens of minutes to hours; RPO minutes	Database replicated continuously to the DR region; application servers kept switched off and scaled up on demand
Warm standby	RTO minutes; RPO seconds to minutes	A scaled-down but running copy of the full stack in the DR region
Active-active	RTO near zero; RPO near zero	Full stack runs in two regions with global load balancing and multi-region database

For this e-learning platform, a reasonable target is an RPO of 15 minutes or less and an RTO of one to two hours. This is achieved with continuous log backup and cross-region replication for the databases (low RPO), versioned object storage with replication for content, and a pilot-light or warm-standby environment in a second region, activated by automation scripts and DNS failover (low RTO). Accidental deletions are handled by restoring a previous object version or a point-in-time database copy, which typically takes minutes.

3.4 Steps to Ensure Business Continuity

- **Step 1:** Classify systems by business impact and set RTO and RPO for each (for example live classes and assessments are critical, archived courses are less so).
- **Step 2:** Detect the incident through monitoring and alerts, and declare it according to a documented incident response plan with named roles.
- **Step 3:** Contain the problem by stopping the faulty deployment or isolating compromised accounts so that further data is not damaged.

- **Step 4:** Restore from the most recent clean version, snapshot or point in time, or fail over to the DR region by promoting the replica and updating DNS.
- **Step 5:** Validate data integrity and application behaviour with automated tests, then redirect users.
- **Step 6:** Communicate with students and staff using status pages and notifications.
- **Step 7:** Hold a post-incident review, fix the root cause (for example restrict delete permissions and add approval workflows) and update the runbooks.
- **Step 8:** Test regularly through scheduled restore tests and disaster-recovery drills, because an untested backup is not a reliable backup.

3.5 Justification

Versioning and immutable backups are chosen because they directly protect against the accidental deletion that caused the problem, while cross-region replication protects against regional failure. The pilot-light model is justified as the best balance between cost and the platform's moderate recovery needs, and it can be upgraded to warm standby around exam periods when availability is more critical.

Question 4: Hybrid Cloud for a Government Agency

4.1 Understanding the Scenario and Key Issues

A government agency holds both sensitive data (citizen identity, tax or security records) and non-sensitive workloads such as public portals and open data. Regulations require some applications and data to stay on-premises, but the agency also wants the elasticity and cost benefits of the cloud for other workloads. The key issues are protecting data in transit between environments, controlling who can access which resource, keeping a consistent security policy in two different environments, meeting audit requirements, and avoiding integration complexity.

4.2 Hybrid Architecture Design

- **Workload placement:** data and applications are classified by sensitivity. Highly sensitive systems and their databases remain in the on-premises data centre or a private cloud. Public-facing portals, development and test environments, analytics on anonymised data, backup and burst capacity run in the public cloud (preferably in a government-approved or in-country region).
- **Private connectivity:** the data centre connects to the cloud through a dedicated private link (AWS Direct Connect, Azure ExpressRoute, Google Cloud Interconnect), with an IPsec site-to-site VPN as an encrypted backup path. Traffic does not cross the public internet.
- **Network design:** a hub-and-spoke virtual network layout with a central transit gateway, private subnets, network segmentation, security groups and next-generation firewalls. Cloud services are reached through private endpoints, and inspection, intrusion detection and DDoS protection are applied at the perimeter.
- **Integration layer:** API gateways and message queues exchange data between environments, with strictly defined interfaces. Only the minimum data needed is sent to the cloud, and sensitive fields are tokenised or masked first.
- **Common platform:** Kubernetes and containers, or cloud-extension platforms such as AWS Outposts, Azure Stack and Anthos, give identical tooling on-premises and in the cloud.

- **Central monitoring and logging:** logs and metrics from both environments are collected into one SIEM for correlation and auditing.

4.3 Identity Management and Encryption

Identity management. A single authoritative identity source, usually the agency's on-premises directory (Active Directory), is federated with the cloud identity service (Azure AD / Entra ID, AWS IAM Identity Center) through SAML or OpenID Connect. This gives single sign-on, so there is one identity per person and access can be revoked in one place. Multi-factor authentication is required, especially for privileged users. Role-based and attribute-based access control implements least privilege, privileged access management provides just-in-time, time-limited administrator access, and workloads use short-lived roles rather than long-term keys. Regular access reviews and logging of all authentication events support audit requirements.

Encryption. Data in transit is protected by TLS 1.2 or higher between applications and by IPsec or MACsec on the connectivity links, and mutual TLS is used between services. Data at rest is encrypted with AES-256 in databases, volumes and object storage. Keys are managed in a key-management service, with sensitive keys generated and stored in on-premises hardware security modules under the agency's sole control (bring-your-own-key or hold-your-own-key), and are rotated regularly. Encryption in use can be supported by confidential computing where required. Secrets such as passwords and certificates are stored in a vault, not in code.

4.3 Challenges in Managing Hybrid Cloud

Challenge	Explanation and mitigation
Security consistency	Policies differ between environments, which creates gaps. Use policy as code, common baselines (such as CIS) and central posture management.
Complexity and skills	Staff must manage two platforms. Use common tooling (Kubernetes, Terraform), automation and training.
Network latency and bandwidth	Chatty applications between on-premises and cloud perform poorly. Keep tightly coupled components together, cache data and size links properly.
Data governance and sovereignty	Data may be copied unintentionally. Use classification, data-loss prevention and region restrictions.
Visibility and monitoring	Fragmented logs hinder incident response. Centralise logging and use unified dashboards.
Cost management	Duplicate infrastructure and egress charges. Use tagging, budgets and regular workload placement reviews.
Integration and compatibility	Legacy systems may not support modern APIs. Use gateways, adapters and phased modernisation.

4.4 Justification

A hybrid model is justified because compliance prevents full migration but the agency still benefits from cloud elasticity for suitable workloads. Dedicated links and local key control are chosen to satisfy sovereignty and confidentiality requirements, and federated identity avoids the weakness of separate credential stores.

Question 5: Multi-Region High Availability for a SaaS Provider

5.1 Understanding the Scenario and Key Issues

A SaaS provider promises customers a service level agreement (SLA), for example 99.99 percent availability, which allows under an hour of downtime in a year. An entire cloud region can fail because of a power, network or software fault, or a faulty configuration change. Single-region deployment, even with multiple Availability Zones, cannot survive such an event. The provider therefore needs a multi-region design with automated failover, data replication and strong monitoring, while managing the cost and complexity of consistency between regions.

5.2 Multi-Region Deployment Strategy

Within each region the application is deployed across at least three Availability Zones, with stateless application servers or containers in auto-scaling groups behind load balancers. Two main patterns are possible between regions:

Pattern	Description	Trade-offs
Active-passive	Primary region serves all traffic; the secondary holds replicated data and a minimal or warm environment that is activated on failure	Lower cost and simpler data consistency; failover takes minutes and some recent data may be lost
Active-active	Two or more regions serve users at the same time, with global routing and multi-region data stores	Near-zero downtime and better latency; higher cost and more complex conflict handling

For a SaaS product with a strict SLA, active-active (or active-passive with a hot standby) is recommended for the stateless application layer, with tenants or users routed to the nearest region. The same infrastructure code deploys identical stacks in every region, and release pipelines roll out changes to one region at a time (progressive delivery) so that a faulty release does not take down all regions simultaneously. Dependencies on a single region's global services are avoided to prevent hidden single points of failure.

5.3 Data Replication and Failover Configuration

Replication. Data is the hardest part. Object storage uses cross-region replication. Relational databases use a primary with cross-region read replicas (for example Aurora Global Database, which typically replicates in under a second) or log shipping. Globally distributed databases such as DynamoDB Global Tables, Cosmos DB or Cloud Spanner allow writes in several regions and resolve conflicts, for example with last-writer-wins or application-defined rules. Caches and queues are replicated or rebuilt in each region. Synchronous replication across regions gives zero data loss but adds latency, so asynchronous replication is normally used, which accepts a small replication lag (a low but non-zero RPO). Replication lag must be monitored, and encryption keys must be available in all regions.

Failover. Global DNS or a global load balancer (Amazon Route 53, Azure Front Door, Google Global Load Balancer or Cloudflare) performs health checks on endpoints in each region. When checks fail, traffic is automatically redirected to healthy regions. Key settings include a low DNS time-to-live (30 to 60 seconds), multiple health-check locations to avoid false positives, and a deliberate failure threshold. Database failover promotes a replica to primary, which can be

automatic or require approval to avoid split-brain situations. Failover procedures are scripted, version-controlled and rehearsed through regular game days and chaos engineering (for example by deliberately shutting down a region in a test) to prove that RTO and RPO targets are met. Capacity in the surviving region is reserved or can auto-scale quickly, because it must absorb the full load of the failed region. Failback to the original region is carried out in a controlled way after resynchronising data.

5.4 Monitoring and Alerting for Proactive Detection

- **Golden signals and SLOs:** collect latency, traffic, errors and saturation for every service, and define service level objectives with error budgets so that alerts relate to customer impact.
- **Health checks and synthetic monitoring:** probes simulate user transactions (login, create record) from several regions every minute to detect failure before customers report it.
- **Infrastructure and replication metrics:** CPU, memory, disk, network, queue depth, database connections and cross-region replication lag, with alerts on abnormal trends.
- **Centralised logs and distributed tracing:** tools such as CloudWatch, Azure Monitor, Prometheus/Grafana, ELK and OpenTelemetry help locate the root cause across microservices.
- **Anomaly detection and predictive alerts:** machine-learning based baselines detect unusual behaviour and forecast resource exhaustion.
- **Alert routing and response:** severity-based alerts go to on-call engineers through paging tools (PagerDuty, Opsgenie), with escalation policies, runbooks and automated remediation (restart, scale out, fail over). Alerts are tuned to avoid fatigue.
- **Independent monitoring:** the monitoring system itself runs outside the monitored region so that it is not lost with it, and a public status page keeps customers informed.

5.5 Justification

A multi-region architecture is the only design that survives the loss of a whole region, which is the explicit requirement. Asynchronous replication with automated DNS failover is chosen as the best balance of performance, cost and recovery. The main risks, namely replication lag, split-brain and untested failover, are addressed by monitoring, controlled promotion procedures and regular failure drills.

Conclusion

The five scenarios show how virtualization and cloud architecture principles are combined to solve real problems. Elasticity, caching and load balancing support large-scale streaming; containers and orchestration provide consistency across clouds; versioning, replication and tested recovery plans protect against data loss; careful identity, encryption and network design make hybrid cloud compliant; and multi-region design with strong monitoring achieves high availability. In every case the design decision has to be justified by the requirements of the scenario, with the associated risks and costs recognised and managed.